

PRAKTIKUM TEKNOLOGI IOT II

LAPORAN PRAKTIK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum Mata Kuliah
Teknologi IOT II



DISUSUN OLEH:

NAMA : Tengku Geisya Gustiana Syah

NPM : 25782028

KELAS :3A

PRODI : TEKNOLOGI REKAYASA INTERNET

Dosen :

Dr. Ir. Septafiansyah Dwi Putra, S. T., M.T.

Agus Ambarwari, S.Pd.,M.kom,

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

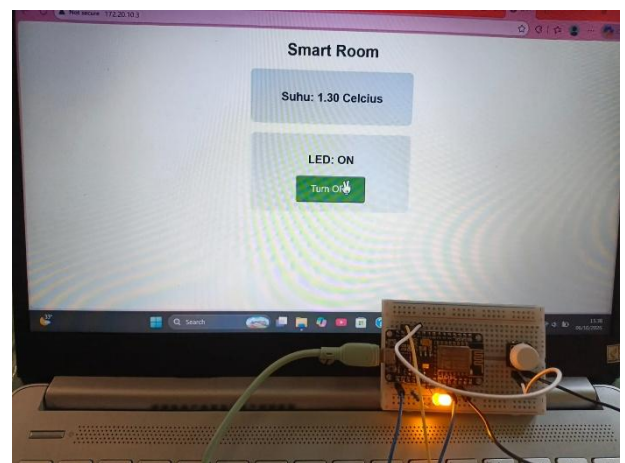
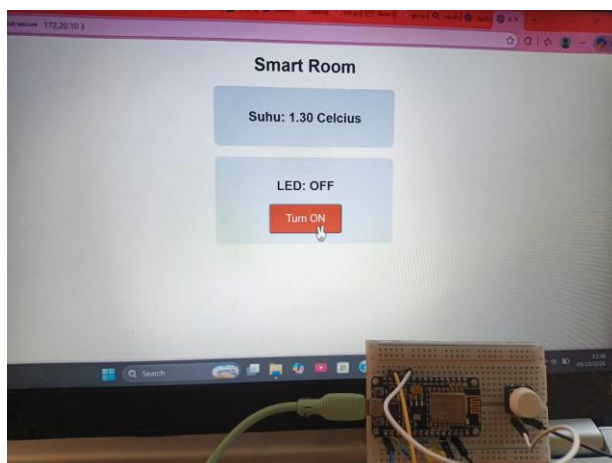
2026

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

```
Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E M...

sketch_oct6a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESPAsyncTCP.h>
3  #include <ESPAsyncWebServer.h>
4  #include <DHT.h>
5
6  const char* ssid = "beabea";
7  const char* password = "matchaenak";
8
9  // Konfigurasi Pin
10 const byte dhtPin = 2;      // D4 (GPIO 2)
11 const byte buttonPin = 4;   // D2 (GPIO 4)
12 const byte ledPin = 12;     // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
22 const unsigned long debounceDelay = 50;
23 unsigned long lastTime = 0;
24
25 // Inisialisasi Async Web Server (port 80) & WebSocket (route /ws)

Output Serial Monitor X
Not connected. Select a board and a port to connect automatically.
IP Address: 172.20.10.3
Client WebSocket #1 terhubung
```



Pada praktikum ini dibuat Real-Time Dashboard menggunakan WebSockets pada NodeMCU ESP8266. Sistem ini menghubungkan LED, push button, dan sensor DHT dengan halaman web. LED dipasang pada pin D6, sensor DHT pada D4, dan push button pada D2.

Pada percobaan ini LED bisa dikontrol melalui dua cara, yaitu menggunakan push button secara langsung dan melalui tombol pada halaman web. Saat tombol fisik ditekan, status LED pada web juga ikut berubah. Begitu juga saat tombol pada web ditekan, LED pada rangkaian akan ikut berubah. Perubahan tersebut dikirim menggunakan WebSockets sehingga tidak perlu melakukan refresh halaman.

Selain mengontrol LED, sistem juga menampilkan suhu dari sensor DHT pada dashboard. Pembacaan suhu dilakukan setiap 3 detik menggunakan `millis()`, sehingga program tidak berhenti seperti jika menggunakan `delay()`. Dengan begitu, tombol dan koneksi WebSocket tetap bisa bekerja saat sensor sedang menunggu waktu pembacaan berikutnya.

Jadi, hasil dari praktikum ini adalah dashboard web yang bisa menampilkan suhu dan status LED secara real-time serta bisa mengontrol LED melalui web maupun tombol fisik.

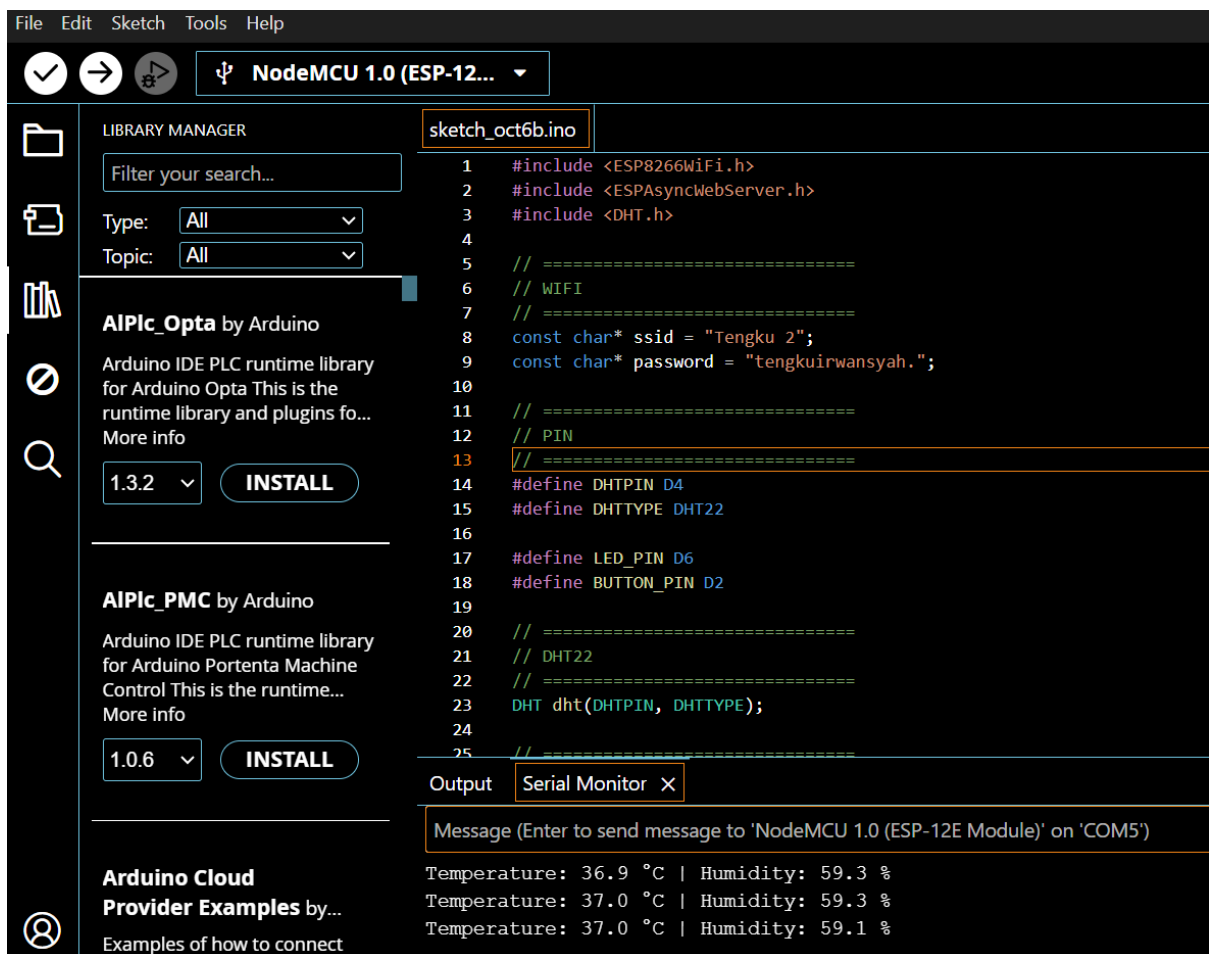
4.6 Latihan

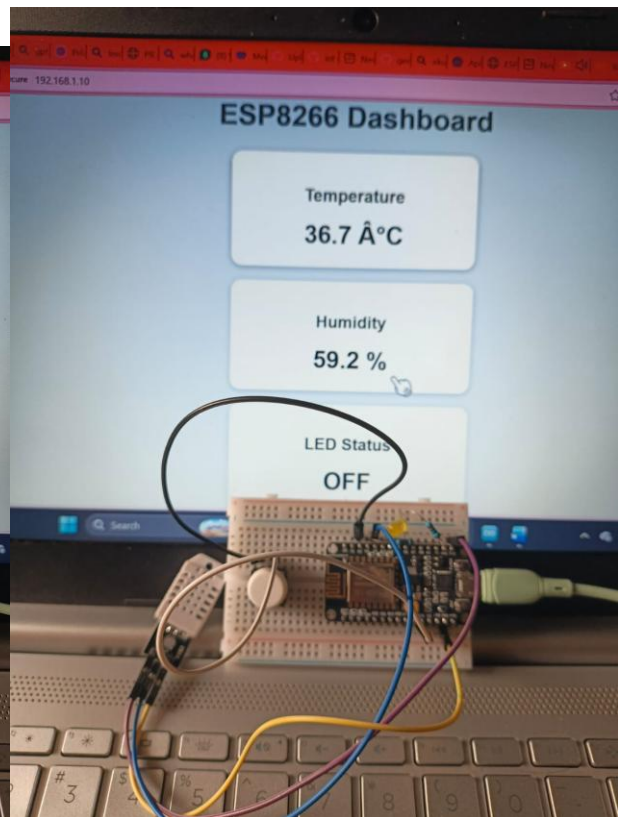
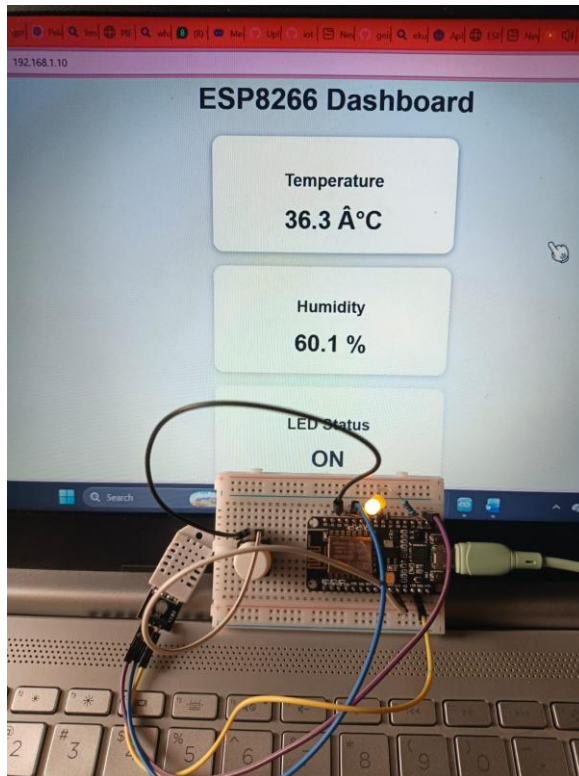
1. Jelaskan mengapa WebSockets lebih hemat bandwidth dan tidak menyebabkan flicker dibandingkan dengan Meta Refresh HTTP.
jawaban: WebSockets lebih hemat bandwidth karena koneksi tetap terbuka sehingga server cukup mengirim data yang diperlukan tanpa memuat ulang seluruh halaman. WebSockets juga tidak menyebabkan flicker karena halaman tidak perlu melakukan refresh secara terus-menerus.
2. Apa yang terjadi jika menggunakan `delay(3000)` pada `loop()` dalam sistem WebSockets?
jawaban: `delay(3000)` membuat program berhenti selama 3 detik. Selama waktu tersebut, proses WebSocket, tombol, dan fungsi lainnya tidak dapat merespons dengan cepat. Karena itu digunakan `millis()` agar pembacaan sensor dapat dilakukan tanpa menghentikan proses lainnya.
3. Identifikasi ID HTML yang menjadi target pada blok JavaScript.
jawaban: ID HTML yang menjadi target JavaScript adalah:
 1. `tempValue` untuk menampilkan nilai suhu.
 2. `ledStatus` untuk menampilkan status LED.
 3. `toggleBtn` untuk mengubah tampilan tombol sesuai kondisi LED.
4. **(Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer

non-blocking millis()). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led": "...", "suhu": "...", "hum": "..."}).

- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Hasilnya:





```
#include <ESP8266WiFi.h>

#include <ESPAsyncWebServer.h>

#include <DHT.h>

// =====

// WIFI

// =====

const char* ssid = "Tengku 2";
const char* password = "tengkuirwansyah.";

// =====

// PIN

// =====

#define DHTPIN D4
#define DHTTYPE DHT22

#define LED_PIN D6
#define BUTTON_PIN D2
```

```

// =====
// DHT22
// =====

DHT dht(DHTPIN, DHTTYPE);

// =====

// SERVER
// =====

AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// =====

// VARIABLE
// =====

bool ledState = false;

String currentTemp = "--";
String currentHum = "--";

unsigned long previousMillis = 0;
const unsigned long interval = 3000;

// =====

// HTML DASHBOARD
// =====

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>

<html>

<head>

```

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

```
<title>ESP8266 Dashboard</title>
```

```
<style>
```

```
body {  
  font-family: Arial, sans-serif;  
  text-align: center;  
  background: #f2f2f2;  
  margin-top: 40px;  
}
```

```
h1 {  
  font-size: 40px;  
}
```

```
.card {  
  background: white;  
  width: 300px;  
  margin: 20px auto;  
  padding: 25px;  
  border-radius: 15px;  
  box-shadow: 0 2px 8px rgba(0,0,0,0.2);  
}
```

```
.value {  
  font-size: 35px;  
  font-weight: bold;  
}
```

```
button {
```

```
padding: 12px 25px;
font-size: 18px;
cursor: pointer;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>ESP8266 Dashboard</h1>
```

```
<!-- TEMPERATURE -->
```

```
<div class="card">
```

```
<h2>Temperature</h2>
```

```
<div class="value">
```

```
<span id="tempValue">--</span> °C
```

```
</div>
```

```
</div>
```

```
<!-- HUMIDITY -->
```

```
<div class="card">
```

```
<h2>Humidity</h2>
```

<div class="value">

-- %

</div>

</div>

<!-- LED -->

<div class="card">

<h2>LED Status</h2>

<div class="value" id="ledStatus">

OFF

</div>

<button onclick="toggleLED()">

Toggle LED

</button>

</div>

<script>

```
var websocket;

// =====
// WEBSOCKET
// =====

function initWebSocket() {

    websocket = new WebSocket(
        "ws://" + window.location.hostname + "/ws"
    );

    websocket.onopen = function() {

        console.log("WebSocket Connected");

    };

    websocket.onclose = function() {

        console.log("WebSocket Disconnected");

        setTimeout(initWebSocket, 2000);

    };

    websocket.onmessage = function(event) {

        var data = JSON.parse(event.data);

        // Temperature
```

```
document.getElementById("tempValue").innerHTML =  
    data.temp;  
  
// Humidity  
  
document.getElementById("humValue").innerHTML =  
    data.hum;  
  
// LED  
  
document.getElementById("ledStatus").innerHTML =  
    data.led;  
  
};  
  
}  
  
// =====  
// TOGGLE LED  
// =====  
  
function toggleLED() {  
  
    if (websocket.readyState === WebSocket.OPEN) {  
  
        websocket.send("toggle");  
  
    }  
  
}
```

```
// Jalankan WebSocket
```

```
window.onload = function() {
```

```
    initWebSocket();
```

```
};
```

```
</script>
```

```
</body>
```

```
</html>
```

```
)rawliteral";
```

```
// =====
```

```
// KIRIM DATA KE BROWSER
```

```
// =====
```

```
void notifyClients() {
```

```
    String message = "{";
```

```
    message += "\"temp\":\",";
```

```
    message += currentTemp;
```

```
    message += "\",\"";
```

```
    message += "\"hum\":\",";
```

```
    message += currentHum;
```

```
    message += "\",\"";
```

```
    message += "\"led\":\",";
```

```
if (ledState) {

    message += "ON";

} else {

    message += "OFF";

}

message += "\n";

message += "}";

ws.textAll(message);

}

// =====
// WEBSOCKET EVENT
// =====

void onWebSocketEvent(
    AsyncWebSocket *server,
    AsyncWebSocketClient *client,
    AwsEventType type,
    void *arg,
    uint8_t *data,
    size_t len
) {

    // CLIENT CONNECT
```

```
if (type == WS_EVT_CONNECT) {

    Serial.println("WebSocket Client Connected");

    String message = "{}";

    message += "\"temp\":\",";
    message += currentTemp;
    message += "\",\"";

    message += "\"hum\":\",";
    message += currentHum;
    message += "\",\"";

    message += "\"led\":\",";

    if (ledState) {

        message += "ON";

    } else {

        message += "OFF";

    }

    message += "\"";

    message += "}";

    client->text(message);
```

```
}
```

```
// CLIENT DISCONNECT
```

```
else if (type == WS_EVT_DISCONNECT) {
```

```
    Serial.println("WebSocket Client Disconnected");
```

```
}
```

```
// DATA FROM BROWSER
```

```
else if (type == WS_EVT_DATA) {
```

```
    String message = "";
```

```
    for (size_t i = 0; i < len; i++) {
```

```
        message += (char)data[i];
```

```
    }
```

```
    if (message == "toggle") {
```

```
        ledState = !ledState;
```

```
        digitalWrite(
```

```
            LED_PIN,
```

```
            ledState ? HIGH : LOW
```

```
        );
```

```
        notifyClients();
```

```
    }

}

}

// =====
// SETUP
// =====

void setup() {

    // SERIAL

    Serial.begin(115200);

    delay(500);

    // =====
    // PIN LED
    // =====

    pinMode(LED_PIN, OUTPUT);

    digitalWrite(LED_PIN, LOW);

    // =====
    // PIN BUTTON
    // =====

    pinMode(BUTTON_PIN, INPUT);
```

```
// =====  
  
// START DHT22  
  
// =====  
  
dht.begin();  
  
// DHT22 membutuhkan waktu setelah mulai  
  
delay(2000);  
  
Serial.println();  
Serial.println("=====");  
Serial.println("ESP8266 DASHBOARD");  
Serial.println("DHT22 + WEBSOCKET");  
Serial.println("=====");  
  
// =====  
  
// WIFI  
  
// =====  
  
WiFi.mode(WIFI_STA);  
  
WiFi.begin(ssid, password);  
  
Serial.print("Menghubungkan ke WiFi");  
  
while (WiFi.status() != WL_CONNECTED) {  
  
    delay(500);  
  
    Serial.print(".");  
}
```

```
}
```

```
Serial.println();
```

```
Serial.println("WiFi Terhubung!");
```

```
Serial.print("IP Address: ");
```

```
Serial.println(WiFi.localIP());
```

```
// =====
```

```
// WEBSOCKET
```

```
// =====
```

```
ws.onEvent(onWebSocketEvent);
```

```
server.addHandler(&ws);
```

```
// =====
```

```
// WEB SERVER
```

```
// =====
```

```
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
```

```
    request->send_P(
```

```
        200,
```

```
        "text/html",
```

```
        index_html
```

```
    );
```

```
});
```

```
server.begin();
```

```
Serial.println("Web Server aktif!");
```

```
Serial.println("Buka IP Address di browser.");
```

```
}
```

```
// =====
```

```
// LOOP
```

```
// =====
```

```
void loop() {
```

```
// =====
```

```
// PUSH BUTTON
```

```
// =====
```

```
static bool lastButtonState = LOW;
```

```
bool buttonState = digitalRead(BUTTON_PIN);
```

```
if (
```

```
    buttonState == HIGH &&
```

```
    lastButtonState == LOW
```

```
) {
```

```
    ledState = !ledState;
```

```
    digitalWrite(
```

```
        LED_PIN,
```

```
        ledState ? HIGH : LOW
```

```
    );
```

```
    notifyClients();

    delay(50);

}

lastButtonState = buttonState;

// =====
// BACA DHT22
// =====

unsigned long currentMillis = millis();

if (
    currentMillis - previousMillis >= interval
) {

    previousMillis = currentMillis;

    // BACA HUMIDITY

    float humidity = dht.readHumidity();

    // BACA TEMPERATURE

    float temperature = dht.readTemperature();

    // =====
    // CEK HASIL PEMBACAAN
    // =====
```

```
if (
    isnan(humidity) ||
    isnan(temperature)
) {

    Serial.println(
        "Gagal membaca DHT22!"
    );

}

else {

    // SIMPAN TEMPERATURE

    currentTemp = String(
        temperature,
        1
    );

    // SIMPAN HUMIDITY

    currentHum = String(
        humidity,
        1
    );

    // =====
    // SERIAL MONITOR
    // =====
```

```
Serial.print("Temperature: ");
```

```
Serial.print(currentTemp);
```

```
Serial.print(" °C");
```

```
Serial.print(" | Humidity: ");
```

```
Serial.print(currentHum);
```

```
Serial.println(" %");
```

```
// =====
```

```
// KIRIM KE DASHBOARD
```

```
// =====
```

```
notifyClients();
```

```
}
```

```
}
```

```
}
```